



Projeto Avaliativo - Módulo 1

Manutenção preditiva para industria 4.0

Pipeline em Python para prever falhas mecânicas, comparar modelos e apoiar monitoramento operacional por dashboard.

10.000

base historica

registros monitorados por sensores

7 fases

entrega central

estrutura do PDF avaliativo

O projeto responde a uma decisão operacional



Antecipar falhas reduz paradas inesperadas e melhora o planejamento de manutenção.

Pergunta do negócio

**Com as leituras atuais de sensores,
a máquina tem risco de falha
mecânica?**

- Alvo binário: `falha_maquina` = 1 ou 0.
- O PDF exige EDA, limpeza, feature engineering, balanceamento, KNN, Arvore e veredito por acurácia.
- O projeto complementar leva a solução para uso prático com modelo salvo e dashboard.

Resultado esperado

Um avaliador deve conseguir executar localmente, entender as escolhas técnicas e ver como o modelo seria usado pela empresa.

A solução foi organizada em duas camadas



O notebook principal cumpre o edital; o complementar mostra a evolução para monitoramento.

1 Notebook principal

Executa as 7 fases do PDF: EDA, limpeza, feature engineering, divisão, balanceamento, KNN, Árvore e comparação final.

2 Notebook complementar

Testa modelos ensemble, usa pipeline com ColumnTransformer e SMOTE, avalia recall/F1 e salva o modelo final.

3 Dashboard Streamlit

Carrega o modelo salvo, estima probabilidade de falha, permite ajustar threshold e exportar previsões.

4 Documentação

README e requirements explicam dependências, caminho local da base e comandos de execução.

A base é tratada antes de qualquer treino

A preparação evita lixo entrando no modelo e preserva sinais industriais relevantes.



- Leitura local em `data/manutencao_preditiva.csv`.
- Tratamento de ausentes com mediana para variáveis numéricas.
- Outliers são analisados por boxplot, mas não removidos automaticamente.
- Feature principal: $\text{potencia_w} = \text{velocidade_rotacao_rpm} \times \text{torque_nm}$.
- No complemento, `delta_temperatura` amplia a leitura térmica.

Fluxo de dados



O controle de vazamento e a regra mais importante



A apresentação deixa claro que o alvo e as causas históricas não entram nas features.

falha_maquina

alvo usado apenas em y
nunca entra em X

5 causas

colunas removidas
twf, hdf, pwf, osf, rnf

udi/id

identificadores fora
sem valor físico direto

Colunas aceitas pelo modelo complementar

- tipo, temperaturas, velocidade, torque e desgaste
- potencia_operacional e delta_temperatura
- validacao impressa: falha_maquina em X: False

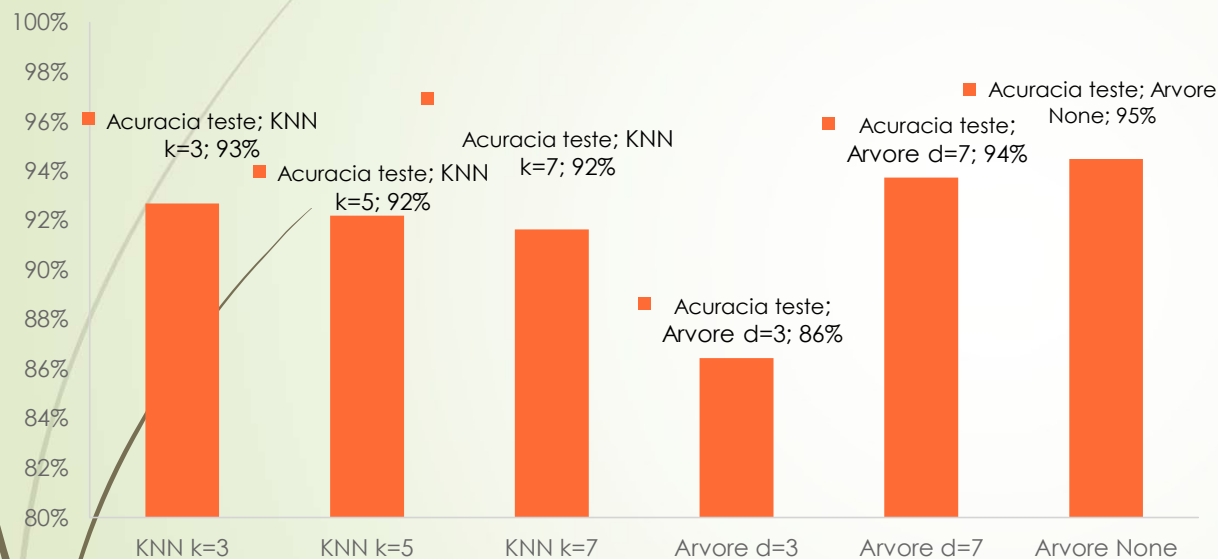
Por que importa?

Sem esse bloqueio, a acurácia poderia parecer alta porque o modelo estaria lendo a resposta pronta.

O notebook principal compara KNN e Arvore de Decisão



A Arvore tem a maior acuracia de teste dentro do escopo obrigatorio.



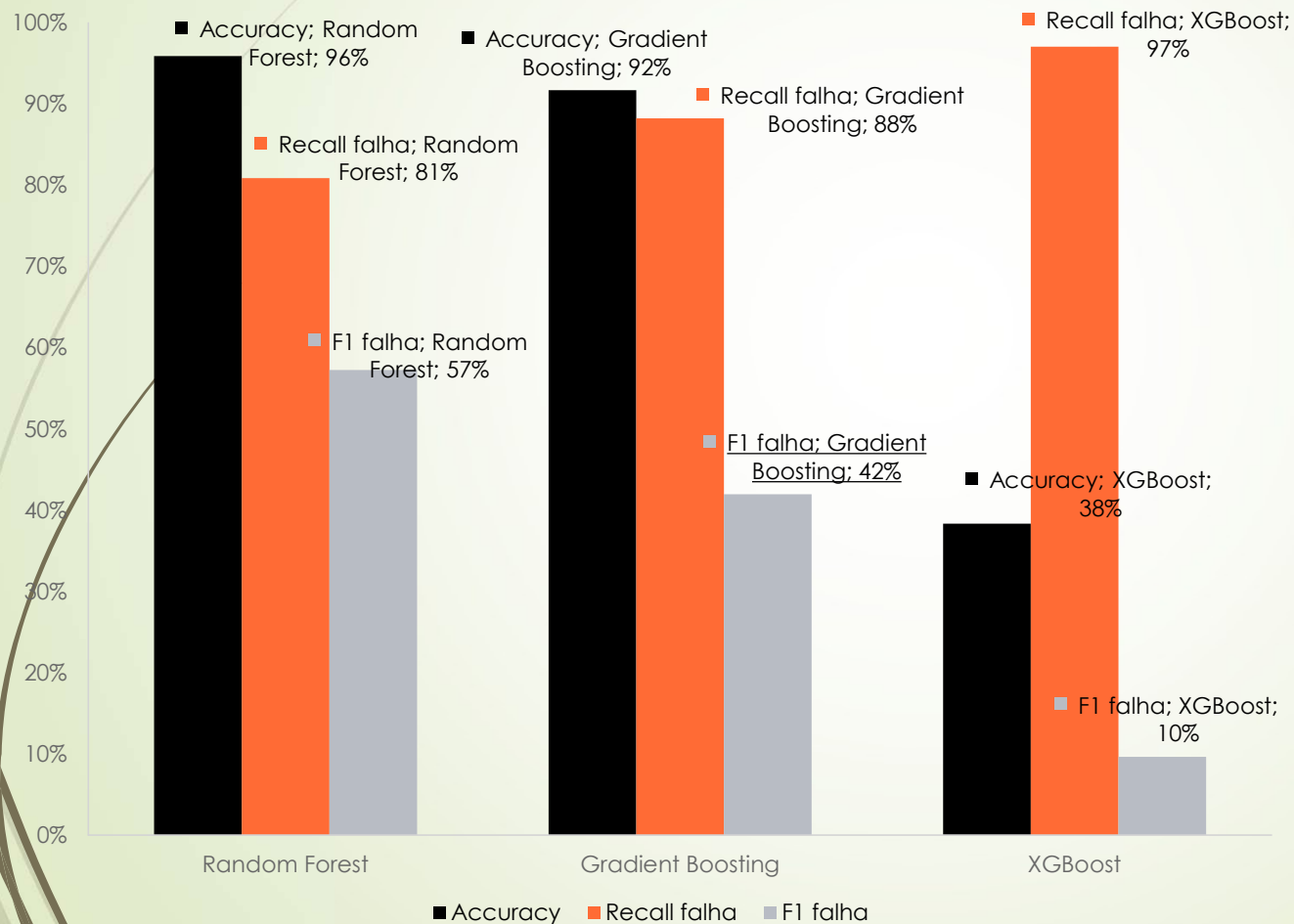
Leitura do resultado

- KNN exige StandardScaler porque calcula distância.
- Arvore fica sem escalonamento, pois separa por regras de corte.
- Melhor resultado obrigatorio: Arvore de Decisao com 0,9450 de acuracia no teste.

O complemento otimiza para detectar falhas raras



Acuracia sozinha pode esconder falhas perdidas; recall e F1 tornam a avaliacao mais industrial.



Random Forest

modelo operacional salvo

threshold 0,50

81%

falhas detectadas

recall da classe falha

Trade-off

O dashboard permite ajustar threshold conforme a tolerancia da empresa a falsos positivos.

O dashboard transforma o modelo em rotina de monitoramento

A interface usa o modelo salvo para priorizar maquinas com maior risco.



1 Entrada

Arquivo local com leituras de sensores e identificadores da maquina.

2 Predicao

Joblib carrega o pipeline e calcula a probabilidade de falha por registro.

3 Decisao

Usuario ajusta threshold, filtra riscos e exporta previsoes em CSV.

Comando de execucao

```
streamlit run dashboard/dashboard_monitoramento.py
```

No VSCode, executar pelo terminal evita rodar o arquivo como Python comum.

A execução local segue quatro comandos simples



A base fica no projeto e não depende de download externo.

1 Instalar dependências

```
pip install -r requirements.txt
```

2 Abrir notebook principal

```
jupyter notebook projeto_manutencao_preditiva.ipynb
```

modelos

3 Gerar modelo complementar

```
jupyter notebook notebook_complementar__dashboard.ipynb
```

4 Abrir dashboard

```
streamlit run dashboard/dashboard_monitoramento.py
```

Base esperada: data/manutencao_preditiva.csv

A mensagem final e simples: o pipeline é auditável e utilizável



A entrega cobre o PDF e mostra uma evolução prática para tomada de decisão.

- O notebook principal segue as fases avaliativas e justifica as escolhas técnicas.
- A modelagem separa **alvo** e **preditores** para evitar vazamento de dados.
- A **Arvore de Decisao** vence no escopo obrigatório por **acuracia**.
- O complemento recomenda **Random Forest** para uso operacional por **recall/F1**.
- O dashboard permite **transformar previsões** em uma rotina de priorização.

Para o video

Objetivo

Execução

Organização

Governança

Modelo final

Dashboard

Essa seqüência responde diretamente aos topicos solicitados no Projeto Avaliativo.